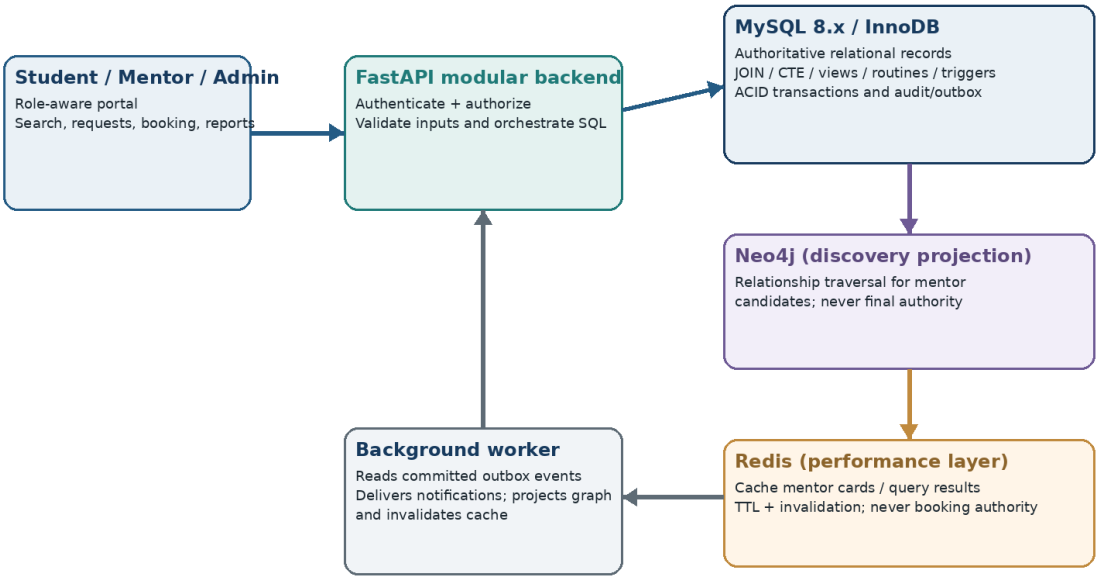
ALUMNICONNECT

SQL QUERY CATALOGUE

Required queries, views, procedures, triggers and analytics

ALUMNI–STUDENT MENTORSHIP & MOCK INTERVIEW PLATFORM

ALUMNICONNECT | SQL REQUEST AND DATA FLOW



MySQL commits business truth. Neo4j and Redis are rebuildable / eventually consistent support stores.

Document control	Specification
Document ID	AC-SQL-001
Version	1.0
Prepared	02 November 2026
Database baseline	MySQL 8.x with InnoDB
Supporting stores	Redis (cache / temporary state); Neo4j (graph discovery projection)
Status	Implementation-oriented design catalogue; physical schema alignment required

Prepared from the supplied AlumniConnect blueprint and companion project specifications

Document guide and implementation status

This catalogue converts the AlumniConnect project requirements into a practical set of MySQL query patterns and database objects. It is intended to guide implementation, testing, SQL demonstrations and the final DBMS project viva. It is not a replacement for the final relational schema: the ER document explicitly treats its entities and attributes as conceptual, and reserves physical names, data types and indexes for the relational-design stage.

Important schema boundary The SQL examples use a consistent snake_case MySQL schema contract defined in Section 3. Some column names are implementation choices because the supplied conceptual ER and PRD do not publish a complete final CREATE TABLE script. Align those names with your final Relational Design Specification before execution.

How requirements are labelled

Label	Meaning
[B] Blueprint / PRD	Explicitly present in the supplied blueprint, PRD, ER model, business rules or workflow documents.
[L] Legacy mini-project	Present in the earlier 12-table Alumni Mentor Allocation mini-project schema.
[R] Recommended implementation	A practical SQL pattern or column/index assumption added to make the design implementable. Confirm configurable policies with the team.
[C] Course alignment	Demonstrates a DBMS concept included in the supplied PES course material.

What this catalogue contains

- A compatibility map from the original 12-table mini-project to the newer AlumniConnect conceptual model.
- A query-ready table and column contract covering profiles, skills, domains, requests, allocations, availability, booking, sessions, feedback, recommendations, audit and outbox records.
- A numbered SQL catalogue covering retrieval, joins, subqueries, set operations, NULL handling, aggregates, CTEs, window functions and full-text search.
- Reusable views, stored functions, transactional stored procedures and audit / validation triggers.
- Index recommendations, EXPLAIN / EXPLAIN ANALYZE demonstrations, transaction considerations and administrator analytics.
- Acceptance checks and a mapping to Units 1–4 of the DBMS course.

Contents

Catalogue contents	Catalogue contents
1. System and database responsibilities	2. Evolution of the database model
3. SQL schema contract and conventions	4. Query catalogue at a glance
5. Core retrieval and join queries	6. Relational operations and subqueries
7. Aggregation and analytics queries	9. Views and safe data presentation
10. Stored functions	11. Stored procedures and transactional operations
12. Triggers and audit enforcement	13. CTEs, window functions and full-text search
14. Indexing and query optimisation	15. Transactions, concurrency and outbox integration
16. Administrator analytics catalogue	17. Verification and acceptance-test matrix
18. DBMS syllabus mapping and implementation order	Appendix A. Source-document traceability
Appendix B. SQL execution checklist	

1. System and database responsibilities

AlumniConnect is specified as a modular monolith with polyglot persistence. One FastAPI application contains the identity, profile, matching, request, allocation, booking, session, feedback and notification modules. MySQL 8.x with InnoDB is the authoritative relational store. Redis is a cache and temporary-state layer. Neo4j is a relationship-oriented projection used to discover and explain mentor candidates.

Store / component	Owns or performs	Must not do
MySQL / InnoDB	Users, profiles, skills, domains, requests, allocations, availability, bookings, sessions, feedback, reviews, outbox and audit data.	Allow a cached or graph result to override current eligibility, capacity or booking state.
Neo4j	Traverses Student, Mentor, Skill, Domain, Company and Department relationships to identify potentially relevant mentors.	Commit allocation or booking decisions; serve as the authoritative source for a profile or skill.
Redis	Caches mentor cards, skill/domain catalogue and recent recommendation results; supports TTL, rate limits and idempotency records.	Decide whether a slot is truly available or bypass a MySQL validation.
FastAPI modules	Authenticate, authorize, validate request data, orchestrate MySQL transactions and present results.	Expose database servers directly to the browser or split coordinated writes into independent commits.
Outbox worker	Processes committed events to deliver notifications, update Neo4j and invalidate Redis keys.	Process uncommitted events or silently discard projection / delivery failures.

1.1 SQL belongs at the centre of business correctness

The SQL catalogue is not just a collection of SELECT statements. The project requires query-based screens and reports, but the most important SQL routines protect cross-table invariants: accepting one pending request creates at most one allocation; booking claims one availability slot; two overlapping appointments cannot be confirmed for the same student or mentor; reviews refer to completed sessions; and notifications are only emitted for committed business changes.

Transaction rule The business row change and the corresponding notification / projection intent must be inserted into notification_outbox in the same MySQL transaction. The worker performs external side effects after commit.

2. Evolution of the database model

The earlier mini-project defines 12 core tables. The later AlumniConnect blueprint expands the model to separate identity, role-specific profiles, domain interests/expertise, recommendation history, mentor availability, bookings, sessions, two kinds of feedback, notification outbox and audit history. Keep the earlier model as a useful migration baseline, but do not treat every old table as the final model.

Earlier 12-table model [L]	Current conceptual model [B]	SQL migration / design note
DEPARTMENT	department	Retain as shared reference data.
STUDENT	user_account + student_profile	Move credentials/contact identity to one account; keep academic attributes on student_profile.
ALUMNI	user_account + mentor_profile	Move credentials/contact identity to one account; keep career/company/capacity on mentor_profile.
COMPANY	company	Retain as shared company reference data.
SKILL	skill	Retain; canonical skill names must not be duplicated.
STUDENT_SKILL	student_skill	Retain many-to-many link and skill-specific attributes.

Earlier 12-table model [L]	Current conceptual model [B]	SQL migration / design note
ALUMNI_SKILL	mentor_skill	Rename to align with Mentor terminology; retain proficiency and experience.
MENTORSHIP_DOMAIN	mentorship_domain	Retain as reusable reference data.
MENTORSHIP_REQUEST	mentorship_request	Canonical statuses become PENDING / ACCEPTED / REJECTED / CANCELLED.
MENTOR_ALLOCATION	mentor_allocation	Accepted relationship; retain status, start/end, match score and reason.
MENTORSHIP_SESSION	mentorship_session + booking	Split planned appointment (booking) from actual meeting/outcome (session).
FEEDBACK	session_feedback + mentor_review	Separate participant session feedback from student rating/review of a mentor.

2.1 Resolve two old/new status differences

Earlier mini-project field	Updated AlumniConnect baseline	Adopted treatment
MENTORSHIP_REQUEST.status = Pending / Allocated / Completed / Cancelled	Pending / Accepted / Rejected / Cancelled	Use Accepted when a mentor approves. Allocation is a separate record. Completed is not a request status; session/allocation lifecycle records completion.
FEEDBACK links to allocation_id and uses given_by_type / given_by_id	SESSION_FEEDBACK links to session_id and actual author; MENTOR_REVIEW links to a completed session and student/mentor	Migrate narrative/participant feedback to session_feedback. Migrate student rating/review to mentor_review only when the source record contains the required provenance. Do not infer missing authors.

Migration caution A legacy feedback rating may not be safely converted to a Mentor Review unless the source can prove which student authored it and which completed session it evaluates. Preserve ambiguous legacy rows in a migration / archive table rather than inventing links.

2.2 Added entities in the expanded model

- role and user_account - shared identity and role-based access.
- student_domain_interest and mentor_domain_expertise - domain preferences and mentor specialisation.
- recommendation_run and recommended_mentor - immutable recommendation history, score, rank and explanation.
- mentor_availability and booking - explicit time slots and planned appointments.
- session_feedback and mentor_review - separate feedback purposes.
- notification_outbox and audit_log - reliable events and historical traceability.

3. SQL schema contract and conventions

The queries below use the names and columns in this section. These are a query-ready reference contract derived from the conceptual entity list, previous mini-project attributes, PRD requirements and workflow documents. They are illustrative SQL identifiers, not a claim that the physical schema has already been deployed. If your final Relational Design Specification uses different names, perform a consistent rename before executing the catalogue.

3.1 Core reference and identity tables

Table	Query-relevant columns (primary key marked PK)	Purpose
role	role_id (PK), role_name	Role master: STUDENT, MENTOR, ADMINISTRATOR.
user_account	user_id (PK), role_id, email, account_status, created_at	One authentication identity; never expose password_hash in public views.

Table	Query-relevant columns (primary key marked PK)	Purpose
department	department_id (PK), department_name, department_code	Academic / alumni department.
company	company_id (PK), company_name, industry, company_type, city	Mentor employer reference data.
student_profile	student_id (PK), user_id (UNIQUE), full_name, department_id, batch_year, current_year, graduation_year, cgpa, career_goal	Academic student attributes.
mentor_profile	mentor_id (PK), user_id (UNIQUE), full_name, department_id, company_id, job_title, current_city, years_experience, mentor_status, capacity, biography, professional_summary	Professional mentor profile and capacity.

3.2 Skills, expertise and domain tables

Table	Query-relevant columns	Purpose
skill	skill_id (PK), skill_name, skill_category, description	Canonical skills.
student_skill	student_id, skill_id, proficiency_level, interest_level, learning_goal; PK(student_id, skill_id)	Student skill and learning interest.
mentor_skill	mentor_id, skill_id, proficiency_level, years_experience, is_primary_skill; PK(mentor_id, skill_id)	Mentor expertise.
mentorship_domain	domain_id (PK), domain_name, description	Career / mentorship domains such as MLOps, Cloud, Data Analytics.
student_domain_interest	student_id, domain_id, priority, interest_level; PK(student_id, domain_id)	Student domain preferences.
mentor_domain_expertise	mentor_id, domain_id, expertise_level; PK(mentor_id, domain_id)	Mentor specialisation areas.

3.3 Lifecycle and operations tables

Table	Query-relevant columns	Purpose
mentorship_request	request_id (PK), student_id, mentor_id, domain_id, objective, message, status, submitted_at, decision_at, decision_reason	Student proposal directed to one mentor.
mentor_allocation	allocation_id (PK), request_id (UNIQUE), student_id, mentor_id, status, match_score, allocation_reason, created_at, start_date, end_date	Accepted mentoring relationship; at most one allocation per request.
mentor_availability	availability_id (PK), mentor_id, start_at, end_at, slot_status, meeting_mode	Mentor-published slot.
booking	booking_id (PK), availability_id, student_id, mentor_id, allocation_id, booking_type, start_at, end_at, status, created_at, cancelled_at	Planned appointment; current ER-aligned default links it to an active allocation.
mentorship_session	session_id (PK), booking_id (UNIQUE), allocation_id, status, session_at, topics, skills_assessed, observations, improvement_areas, next_steps, recommended_preparation, completed_at	Actual offline meeting and outcome.
session_feedback	feedback_id (PK), session_id, author_user_id, communication_rating, helpfulness_rating, technical_preparation_rating, comments, created_at	Structured feedback authored by an actual participant.
mentor_review	review_id (PK), session_id, student_id, mentor_id, rating, review_text, created_at	Student review of mentor after completed session.

3.4 Recommendation, audit and event tables

Table	Query-relevant columns	Purpose
recommendation_run	run_id (PK), student_id, criteria_version, input_snapshot, filters_json, generated_at	One recommendation execution and its criteria snapshot.
recommended_mentor	recommendation_id (PK), run_id, mentor_id, rank, score, score_breakdown, explanation, selected_flag	Candidate, ranking and reason within a run.
notification_outbox	event_id (PK), event_type, aggregate_type, aggregate_id, payload, delivery_status, created_at, processed_at	Durable event to be processed after transaction commit.
audit_log	audit_id (PK), entity_type, record_id, action, actor_user_id, old_values, new_values, occurred_at	Trace important status and administrative changes.

Open design decision - one-off mock interviews The current Conceptual ER model states that each Mentorship Session belongs to a Mentor Allocation. Therefore the default SQL booking routine requires an active allocation for any booking that will produce a session, including a mock interview. If the team wants one-off mock interviews without a mentorship allocation, revise the ER / schema first: allow booking.allocation_id and mentorship_session.allocation_id to be NULL, while keeping the booking-to-session relationship authoritative.

3.5 Canonical status vocabulary

Entity	Status values used in this catalogue	Notes
user_account	ACTIVE, LOCKED, DISABLED	Account status is checked before protected actions.
mentor_profile	AVAILABLE, UNAVAILABLE	Only AVAILABLE mentors are candidates for new matches.
mentorship_request	PENDING, ACCEPTED, REJECTED, CANCELLED	Rejected/cancelled requests remain in history.
mentor_allocation	ACTIVE, COMPLETED, CANCELLED	Counts toward active mentoring capacity only when ACTIVE.
mentor_availability	AVAILABLE, BOOKED, WITHDRAWN	MySQL decides current availability.
booking	CONFIRMED, CANCELLED, COMPLETED, NO_SHOW	A booking represents a planned appointment and its later disposition.
mentorship_session	SCHEDULED, COMPLETED, CANCELLED, NO_SHOW	Session status records the actual meeting outcome.
notification_outbox	PENDING, PROCESSED, FAILED	Worker retry lifecycle.

3.6 MySQL and naming conventions

- Target MySQL 8.x with InnoDB. Use foreign keys, NOT NULL, UNIQUE and CHECK constraints wherever they express row-level invariants.
- Use uppercase status values consistently in stored values and SQL predicates. Do not mix Pending, PENDING and pending in application code.
- Store timestamps consistently (prefer UTC at the persistence boundary); format to the user time zone in the application.
- Use DECIMAL for scores and ratings; never store numeric ratings as VARCHAR.
- All code examples are read/write templates. Run DDL and stored routines against a test database first.
- The SQL uses MySQL user variables such as @student_id for Workbench demonstration. In FastAPI, bind values through prepared statements rather than concatenating user input.
- The procedure samples use BIGINT auto-increment identifiers with LAST_INSERT_ID(). If the physical schema uses UUIDs or application-generated keys, pass the created IDs into the routine and remove LAST_INSERT_ID() calls.
- The examples assume primary and foreign keys exist. Add/rename columns only during the final physical-schema stage.

Security note Exclude credentials from public views; parameterize user input.

4. Query catalogue at a glance

The blueprint specifically asks for meaningful business queries rather than an arbitrary query count. The list below follows the project questions in the original mini-project and expands them to cover the latest PRD, workflows and course outcomes.

IDs	Area	Business questions	Main SQL concepts
Q01–Q06	Identity and profile retrieval	Student / mentor profile, safe directory, company/department, skills, NULL / incomplete profiles	SELECT, JOIN, LEFT JOIN, WHERE
Q07–Q14	Discovery and recommendation support	Find mentors by shared skills/domain; combine candidate routes; explain matching inputs	JOIN, UNION, GROUP BY, EXISTS
Q15–Q20	Requests, allocation and history	Pending requests, student request history, active mentees, students without allocation	JOIN, correlated NOT EXISTS, subqueries
Q21–Q28	Session, feedback and review analytics	Completed sessions, mentor quality, department/domain metrics, stale activity	Aggregate, HAVING, CASE, date functions
Q29–Q32	Advanced analysis	Request-to-session funnel, ranking by domain, full-text search, latest session per student	CTE, window functions, FULLTEXT
V01–V04	Views	Public directory, rating summary, student progress, admin-safe request funnel	CREATE VIEW
SP01–SP05	Stored procedures	Accept request, create booking, cancel booking, record session outcome, submit review	Transactions, row locks, SIGNAL
TR01–TR04	Triggers	Request status validation, audit logging, session transition validation, review guard	BEFORE / AFTER row triggers

4.1 Roles that use the query catalogue

Actor	Common reads / writes
Student / Mentee	Mentor discovery, skill/domain matching, request history, available slots, own bookings, session history and review eligibility.
Alumni / Mentor	Profile and skill management, pending request inbox, active allocations, schedule, session recording and received review summary.
Administrator	Catalogue maintenance, user status oversight, request/allocation/session funnel, mentor capacity and rating / domain reports.
Background worker	Read committed outbox events and mark delivery / projection status.

5. Core retrieval and join queries

These queries power the normal application screens. Keep their result sets narrow and role-specific. Views are used later to provide stable, privacy-safe presentation surfaces.

Q01 Student profile with department

Purpose: Display a student profile and its department for a student dashboard.

DBMS concepts: [C] SELECT, INNER JOIN, primary/foreign-key navigation.

```
SELECT
    sp.student_id,
    sp.full_name,
    d.department_name,
    sp.batch_year,
    sp.current_year,
    sp.graduation_year,
    sp.cgpa,
    sp.career_goal
FROM student_profile AS sp
JOIN department AS d
  ON d.department_id = sp.department_id
WHERE sp.student_id = @student_id;
```

Implementation note: The query intentionally does not return user authentication fields.

Q02 Public mentor directory

Purpose: Show discoverable active mentors with company and department context.

DBMS concepts: [B/C] Multi-table JOIN; filtering; safe projection of public fields.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    mp.job_title,
    mp.current_city,
    mp.years_experience,
    c.company_name,
    c.industry,
    d.department_name
FROM mentor_profile AS mp
JOIN user_account AS ua
  ON ua.user_id = mp.user_id
LEFT JOIN company AS c
  ON c.company_id = mp.company_id
LEFT JOIN department AS d
  ON d.department_id = mp.department_id
WHERE ua.account_status = 'ACTIVE'
  AND mp.mentor_status = 'AVAILABLE'
ORDER BY mp.full_name;
```

Implementation note: If a mentor has no current company, the LEFT JOIN still allows the profile to appear with a NULL company field.

Q03 Mentor skills and proficiency

Purpose: List a mentor's registered skills and expertise level.

DBMS concepts: [C] JOIN across a many-to-many associative table.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    s.skill_name,
    ms.proficiency_level,
    ms.years_experience,
    ms.is_primary_skill
FROM mentor_profile AS mp
JOIN mentor_skill AS ms
  ON ms.mentor_id = mp.mentor_id
JOIN skill AS s
  ON s.skill_id = ms.skill_id
WHERE mp.mentor_id = @mentor_id
ORDER BY ms.is_primary_skill DESC,
         s.skill_name;
```

Implementation note: The skill master is stored once; mentor_skill stores mentor-specific proficiency and experience.

Q04 Student skills and learning goals

Purpose: Display student skills, current proficiency and learning goals.

DBMS concepts: [C] JOIN and associative-entity retrieval.

```
SELECT
    sp.student_id,
    sp.full_name,
    s.skill_name,
    ss.proficiency_level,
    ss.interest_level,
    ss.learning_goal
```

```
FROM student_profile AS sp
JOIN student_skill AS ss
  ON ss.student_id = sp.student_id
JOIN skill AS s
  ON s.skill_id = ss.skill_id
WHERE sp.student_id = @student_id
ORDER BY ss.interest_level DESC,
         s.skill_name;
```

Q05 Mentor expertise by domain

Purpose: Show which mentorship domains a mentor specialises in.

DBMS concepts: [C] JOIN through mentor_domain_expertise.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    md.domain_name,
    mde.expertise_level
FROM mentor_profile AS mp
JOIN mentor_domain_expertise AS mde
  ON mde.mentor_id = mp.mentor_id
JOIN mentorship_domain AS md
  ON md.domain_id = mde.domain_id
WHERE mp.mentor_id = @mentor_id
ORDER BY md.domain_name;
```

Q06 Incomplete mentor profiles (NULL handling)

Purpose: Find available mentors whose company or professional fields are missing so an administrator can improve directory quality.

DBMS concepts: [C] NULL semantics, IS NULL, COALESCE.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    mp.job_title,
    mp.current_city,
    mp.years_experience,
    CASE
      WHEN mp.company_id IS NULL THEN 'COMPANY_MISSING'
      WHEN mp.job_title IS NULL OR mp.job_title = '' THEN 'JOB_TITLE_MISSING'
      WHEN mp.biography IS NULL OR mp.biography = '' THEN 'BIO_MISSING'
      ELSE 'COMPLETE'
    END AS completeness_flag
FROM mentor_profile AS mp
WHERE mp.mentor_status = 'AVAILABLE'
  AND (
       mp.company_id IS NULL
       OR mp.job_title IS NULL OR mp.job_title = ''
       OR mp.biography IS NULL OR mp.biography = ''
  )
ORDER BY mp.mentor_id;
```

Implementation note: Use IS NULL, not = NULL. An empty string and SQL NULL are different values.

6. Discovery, relational operations and subqueries

The older mini-project asks for suitable alumni based on skill overlap. The newer blueprint adds domain relevance, relevant experience, career relevance, mentor quality and capacity. Neo4j supplies graph-based candidate paths, while MySQL revalidates eligibility, current capacity and quality before the final result. The following SQL queries support the relational side of discovery and reporting.

Q07 Mentor candidates by shared skills

Purpose: Find mentors sharing one or more skills with a student, ordered by number of distinct matching skills.

DBMS concepts: [B/C] JOIN, COUNT(DISTINCT), GROUP BY, ORDER BY.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    COUNT(DISTINCT ss.skill_id) AS matched_skill_count,
    GROUP_CONCAT(DISTINCT s.skill_name
                 ORDER BY s.skill_name SEPARATOR ', ') AS matched_skills
FROM student_skill AS ss
JOIN mentor_skill AS ms
```

```
  ON ms.skill_id = ss.skill_id
JOIN mentor_profile AS mp
  ON mp.mentor_id = ms.mentor_id
JOIN user_account AS ua
  ON ua.user_id = mp.user_id
JOIN skill AS s
  ON s.skill_id = ss.skill_id
WHERE ss.student_id = @student_id
  AND ua.account_status = 'ACTIVE'
  AND mp.mentor_status = 'AVAILABLE'
GROUP BY mp.mentor_id, mp.full_name
ORDER BY matched_skill_count DESC,
         mp.years_experience DESC;
```

Implementation note: This is a SQL fallback / relational ranking query. A production recommendation may add domain and career dimensions from the stored recommendation score.

Q08 Mentors in a student-preferred domain

Purpose: Retrieve active mentors whose expertise matches at least one domain of interest for a student.

DBMS concepts: [B/C] JOIN, DISTINCT and many-to-many domain mapping.

```
SELECT DISTINCT
    mp.mentor_id,
    mp.full_name,
    md.domain_name,
    mde.expertise_level
FROM student_domain_interest AS sdi
JOIN mentor_domain_expertise AS mde
  ON mde.domain_id = sdi.domain_id
JOIN mentor_profile AS mp
  ON mp.mentor_id = mde.mentor_id
JOIN user_account AS ua
  ON ua.user_id = mp.user_id
JOIN mentorship_domain AS md
  ON md.domain_id = sdi.domain_id
WHERE sdi.student_id = @student_id
  AND ua.account_status = 'ACTIVE'
  AND mp.mentor_status = 'AVAILABLE'
ORDER BY md.domain_name,
         mp.full_name;
```

Q09 Combine skill-based and domain-based mentor candidates (UNION)

Purpose: Combine two independent candidate retrieval routes without returning the same mentor twice.

DBMS concepts: [C] Set operation UNION; de-duplication.

```
SELECT
    candidates.mentor_id,
    mp.full_name,
    mp.job_title,
    mp.years_experience
FROM (
    SELECT ms.mentor_id
    FROM student_skill AS ss
    JOIN mentor_skill AS ms
      ON ms.skill_id = ss.skill_id
    WHERE ss.student_id = @student_id

    UNION

    SELECT mde.mentor_id
    FROM student_domain_interest AS sdi
    JOIN mentor_domain_expertise AS mde
      ON mde.domain_id = sdi.domain_id
    WHERE sdi.student_id = @student_id
) AS candidates
JOIN mentor_profile AS mp
  ON mp.mentor_id = candidates.mentor_id
JOIN user_account AS ua
  ON ua.user_id = mp.user_id
WHERE ua.account_status = 'ACTIVE'
  AND mp.mentor_status = 'AVAILABLE'
ORDER BY mp.years_experience DESC,
         mp.full_name;
```

Implementation note: UNION removes duplicates. UNION ALL is faster but should only be used when duplicate rows are impossible or intentionally retained.

Q10 Find students without an active allocation (correlated NOT EXISTS)

Purpose: Identify students who currently have no active mentor allocation.

DBMS concepts: [C] Correlated subquery / NOT EXISTS.

```
SELECT
    sp.student_id,
    sp.full_name,
    d.department_name,
    sp.career_goal
FROM student_profile AS sp
LEFT JOIN department AS d
  ON d.department_id = sp.department_id
WHERE NOT EXISTS (
    SELECT 1
    FROM mentor_allocation AS ma
    WHERE ma.student_id = sp.student_id
      AND ma.status = 'ACTIVE'
)
ORDER BY sp.student_id;
```

Implementation note: NOT EXISTS avoids counting duplicate joined allocation rows and is a good fit for “no current allocation” queries.

Q11 Mentors with above-average experience in their domain

Purpose: Compare a mentor’s experience with the average experience of mentors in the same specialisation domain.

DBMS concepts: [C] Correlated subquery with domain correlation.

```
SELECT DISTINCT
    mp.mentor_id,
    mp.full_name,
    md.domain_name,
    mp.years_experience
FROM mentor_profile AS mp
JOIN mentor_domain_expertise AS mde
  ON mde.mentor_id = mp.mentor_id
JOIN mentorship_domain AS md
  ON md.domain_id = mde.domain_id
WHERE mp.years_experience > (
    SELECT AVG(mp2.years_experience)
    FROM mentor_profile AS mp2
    JOIN mentor_domain_expertise AS mde2
      ON mde2.mentor_id = mp2.mentor_id
    WHERE mde2.domain_id = mde.domain_id
      AND mp2.mentor_status = 'AVAILABLE'
)
  AND mp.mentor_status = 'AVAILABLE'
ORDER BY md.domain_name,
         mp.years_experience DESC;
```

Implementation note: If one mentor specialises in multiple domains, the mentor may appear once per domain. DISTINCT removes exact duplicate projections; retaining domain_name makes each domain-specific comparison visible.

Q12 Alumni / mentors with a particular skill at Expert level

Purpose: Find mentors whose skill proficiency is Expert for a selected skill.

DBMS concepts: [L/C] JOIN, equality filtering and parameterized predicates.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    s.skill_name,
    ms.proficiency_level,
    ms.years_experience
FROM mentor_skill AS ms
JOIN mentor_profile AS mp
  ON mp.mentor_id = ms.mentor_id
JOIN skill AS s
  ON s.skill_id = ms.skill_id
WHERE s.skill_name = @skill_name
  AND UPPER(ms.proficiency_level) = 'EXPERT'
  AND mp.mentor_status = 'AVAILABLE'
ORDER BY ms.years_experience DESC;
```

Implementation note: The legacy schema used ALUMNI_SKILL; the updated model uses mentor_skill.

Q13 Mentors currently mentoring the most students

Purpose: Rank mentors by current active allocation load and show remaining capacity.

DBMS concepts: [L/C] LEFT JOIN aggregate, COALESCE, GROUP BY.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    mp.capacity,
    COUNT(ma.allocation_id) AS active_mentees,
    mp.capacity - COUNT(ma.allocation_id) AS remaining_capacity
FROM mentor_profile AS mp
LEFT JOIN mentor_allocation AS ma
  ON ma.mentor_id = mp.mentor_id
 AND ma.status = 'ACTIVE'
GROUP BY mp.mentor_id, mp.full_name, mp.capacity
ORDER BY active_mentees DESC,
         remaining_capacity ASC;
```

Implementation note: This is a reporting query only. A stored procedure must lock and re-check capacity before accepting a request.

Q14 Find mentors with no reviews yet

Purpose: Identify available mentors who have not received any mentor review.

DBMS concepts: [C] Anti-join with NOT EXISTS.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    mp.job_title,
    mp.years_experience
FROM mentor_profile AS mp
WHERE mp.mentor_status = 'AVAILABLE'
  AND NOT EXISTS (
      SELECT 1
      FROM mentor_review AS mr
      WHERE mr.mentor_id = mp.mentor_id
  )
ORDER BY mp.full_name;
```

Implementation note: Do not treat “no review” as a rating of zero. Display it as NOT YET RATED.

7. Requests, allocations, bookings and session history

Q15 Mentor pending-request inbox

Purpose: Show pending requests for the signed-in mentor, including student details and requested domain.

DBMS concepts: [B/C] Multi-table JOIN and lifecycle status filter.

```
SELECT
    mr.request_id,
    mr.submitted_at,
    sp.student_id,
    sp.full_name AS student_name,
    d.department_name,
    md.domain_name,
    mr.objective,
    mr.message
FROM mentorship_request AS mr
JOIN student_profile AS sp
  ON sp.student_id = mr.student_id
LEFT JOIN department AS d
  ON d.department_id = sp.department_id
JOIN mentorship_domain AS md
  ON md.domain_id = mr.domain_id
WHERE mr.mentor_id = @mentor_id
  AND mr.status = 'PENDING'
ORDER BY mr.submitted_at ASC;
```

Q16 Student request and decision history

Purpose: Show all requests by one student, including accepted/rejected/cancelled decisions.

DBMS concepts: [B/C] JOIN and ORDER BY history.

```
SELECT
    mr.request_id,
```

```
    mp.full_name AS mentor_name,
    md.domain_name,
    mr.objective,
    mr.status,
    mr.submitted_at,
    mr.decision_at,
    mr.decision_reason
FROM mentorship_request AS mr
JOIN mentor_profile AS mp
  ON mp.mentor_id = mr.mentor_id
JOIN mentorship_domain AS md
  ON md.domain_id = mr.domain_id
WHERE mr.student_id = @student_id
ORDER BY mr.submitted_at DESC;
```

Implementation note: Rejected and cancelled requests remain queryable. Do not delete them merely because they did not result in an allocation.

Q17 Student mentorship history with sessions

Purpose: Display allocation and session outcomes for the student, including upcoming scheduled sessions.

DBMS concepts: [B/C] JOIN across request / allocation / session / booking.

```
SELECT
    ma.allocation_id,
    mp.full_name AS mentor_name,
    ma.status AS allocation_status,
    ms.session_id,
    ms.session_at,
    ms.status AS session_status,
    ms.topics,
    ms.next_steps
FROM mentor_allocation AS ma
JOIN mentor_profile AS mp
  ON mp.mentor_id = ma.mentor_id
LEFT JOIN mentorship_session AS ms
  ON ms.allocation_id = ma.allocation_id
WHERE ma.student_id = @student_id
ORDER BY ma.created_at DESC,
         ms.session_at DESC;
```

Implementation note: The LEFT JOIN preserves the allocation when no session has been recorded yet.

Q18 Completed mentorship sessions per mentor

Purpose: Count completed sessions for each mentor.

DBMS concepts: [L/C] Aggregate COUNT with joins and GROUP BY.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    COUNT(ms.session_id) AS completed_sessions
FROM mentor_profile AS mp
LEFT JOIN mentor_allocation AS ma
  ON ma.mentor_id = mp.mentor_id
LEFT JOIN mentorship_session AS ms
  ON ms.allocation_id = ma.allocation_id
 AND ms.status = 'COMPLETED'
GROUP BY mp.mentor_id, mp.full_name
ORDER BY completed_sessions DESC;
```

Implementation note: COUNT(session_id) returns zero for mentors with no completed sessions.

Q19 Students with no recent completed session

Purpose: Find allocated students who have not completed a session in the last 90 days.

DBMS concepts: [L/C] NOT EXISTS with date filtering.

```
SELECT
    sp.student_id,
    sp.full_name,
    mp.full_name AS mentor_name,
    ma.allocation_id,
    ma.start_date
FROM mentor_allocation AS ma
JOIN student_profile AS sp
  ON sp.student_id = ma.student_id
JOIN mentor_profile AS mp
  ON mp.mentor_id = ma.mentor_id
WHERE ma.status = 'ACTIVE'
  AND NOT EXISTS (
```

```
      SELECT 1
      FROM mentorship_session AS ms
      WHERE ms.allocation_id = ma.allocation_id
        AND ms.status = 'COMPLETED'
        AND ms.session_at >= DATE_SUB(CURRENT_TIMESTAMP, INTERVAL 90 DAY)
  )
ORDER BY ma.start_date;
```

Implementation note: Use the agreed reporting window as a parameter in the application if admins need 30/60/90-day choices.

Q20 Booking calendar for a mentor

Purpose: Retrieve confirmed bookings for a mentor over a time window.

DBMS concepts: [B/C] Range predicates; booking is a planned appointment.

```
SELECT
    b.booking_id,
    b.booking_type,
    b.start_at,
    b.end_at,
    b.status,
    sp.student_id,
    sp.full_name AS student_name,
    ma.allocation_id
FROM booking AS b
JOIN student_profile AS sp
  ON sp.student_id = b.student_id
LEFT JOIN mentor_allocation AS ma
  ON ma.allocation_id = b.allocation_id
WHERE b.mentor_id = @mentor_id
  AND b.start_at >= @period_start
  AND b.start_at <  @period_end
  AND b.status IN ('CONFIRMED','COMPLETED','NO_SHOW')
ORDER BY b.start_at;
```

Implementation note: Use half-open time ranges [start, end) to avoid overlap at midnight or adjacent reporting periods.

8. Aggregation and operational analytics queries

The original mini-project lists reporting questions such as request counts by department and domain, mentor load, average rating, completed sessions, students without recent sessions and domains with the highest request volume. The following queries implement those questions against the updated model.

Q21 Requests by department and mentorship domain

Purpose: Count requests by the student's department and the requested domain.

DBMS concepts: [L/C] JOIN, COUNT, GROUP BY.

```
SELECT
    d.department_name,
    md.domain_name,
    COUNT(*) AS request_count
FROM mentorship_request AS mr
JOIN student_profile AS sp
  ON sp.student_id = mr.student_id
JOIN department AS d
  ON d.department_id = sp.department_id
JOIN mentorship_domain AS md
  ON md.domain_id = mr.domain_id
GROUP BY d.department_name, md.domain_name
ORDER BY request_count DESC,
         d.department_name,
         md.domain_name;
```

Q22 Mentors with highest average review rating

Purpose: Show mentors above a minimum review count and rating threshold.

DBMS concepts: [L/C] AVG, COUNT, GROUP BY and HAVING.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    COUNT(mr.review_id) AS review_count,
    ROUND(AVG(mr.rating), 2) AS average_rating
FROM mentor_profile AS mp
JOIN mentor_review AS mr
  ON mr.mentor_id = mp.mentor_id
```

```
GROUP BY mp.mentor_id, mp.full_name
HAVING COUNT(mr.review_id) >= 3
   AND AVG(mr.rating) >= 4.00
ORDER BY average_rating DESC,
         review_count DESC;
```

Implementation note: The minimum sample count (3) and rating threshold (4.00) are illustrative report parameters; the blueprint does not prescribe them.

Q23 Average mentor rating by department

Purpose: Calculate average review score for mentors grouped by their alumni department.

DBMS concepts: [L/C] AVG, LEFT JOIN, GROUP BY, NULL handling.

```
SELECT
    d.department_name,
    COUNT(mr.review_id) AS review_count,
    ROUND(AVG(mr.rating), 2) AS average_rating
FROM department AS d
JOIN mentor_profile AS mp
  ON mp.department_id = d.department_id
LEFT JOIN mentor_review AS mr
  ON mr.mentor_id = mp.mentor_id
GROUP BY d.department_id, d.department_name
ORDER BY average_rating DESC,
         review_count DESC;
```

Implementation note: Departments with no reviews have NULL average_rating. Do not replace NULL with zero unless the dashboard labels it as “no reviews”.

Q24 Mentorship domains with highest request volume

Purpose: Find the most requested mentorship domains.

DBMS concepts: [L/C] LEFT JOIN, COUNT, GROUP BY.

```
SELECT
    md.domain_id,
    md.domain_name,
    COUNT(mr.request_id) AS total_requests
FROM mentorship_domain AS md
LEFT JOIN mentorship_request AS mr
  ON mr.domain_id = md.domain_id
GROUP BY md.domain_id, md.domain_name
ORDER BY total_requests DESC,
         md.domain_name;
```

Q25 Monthly session completion report

Purpose: Count completed sessions by calendar month.

DBMS concepts: [C] DATE_FORMAT, COUNT and GROUP BY.

```
SELECT
    DATE_FORMAT(ms.session_at, '%Y-%m') AS session_month,
    COUNT(*) AS completed_sessions
FROM mentorship_session AS ms
WHERE ms.status = 'COMPLETED'
  AND ms.session_at >= @period_start
  AND ms.session_at <  @period_end
GROUP BY DATE_FORMAT(ms.session_at, '%Y-%m')
ORDER BY session_month;
```

Implementation note: The date range remains index-friendly because the WHERE clause filters the raw timestamp column.

Q26 Booking cancellation and no-show rates

Purpose: Calculate confirmed/completed, cancelled and no-show totals by month.

DBMS concepts: [R/C] Conditional aggregation and NULLIF.

```
SELECT
    DATE_FORMAT(b.start_at, '%Y-%m') AS booking_month,
    COUNT(*) AS total_bookings,
    SUM(b.status = 'CANCELLED') AS cancelled_count,
    SUM(b.status = 'NO_SHOW') AS no_show_count,
    SUM(b.status IN ('CONFIRMED','COMPLETED')) AS attended_or_upcoming_count,
    ROUND(100.0 * SUM(b.status = 'CANCELLED') /
          NULLIF(COUNT(*), 0), 2) AS cancellation_pct,
    ROUND(100.0 * SUM(b.status = 'NO_SHOW') /
          NULLIF(COUNT(*), 0), 2) AS no_show_pct
FROM booking AS b
```

```
WHERE b.start_at >= @period_start
  AND b.start_at <  @period_end
GROUP BY DATE_FORMAT(b.start_at, '%Y-%m')
ORDER BY booking_month;
```

Implementation note: MySQL treats boolean expressions as 1 / 0 in numeric contexts. NULLIF protects percentage calculations from divide-by-zero.

Q27 Mentor capacity utilisation

Purpose: Show active mentees, maximum capacity and utilisation percentage.

DBMS concepts: [R/C] LEFT JOIN, aggregate and scalar arithmetic.

```
SELECT
    mp.mentor_id,
    mp.full_name,
    mp.capacity,
    COUNT(ma.allocation_id) AS active_mentees,
    ROUND(100.0 * COUNT(ma.allocation_id) /
          NULLIF(mp.capacity, 0), 2) AS capacity_utilisation_pct
FROM mentor_profile AS mp
LEFT JOIN mentor_allocation AS ma
  ON ma.mentor_id = mp.mentor_id
 AND ma.status = 'ACTIVE'
GROUP BY mp.mentor_id, mp.full_name, mp.capacity
ORDER BY capacity_utilisation_pct DESC;
```

Implementation note: This is analytics, not a substitute for a row lock and capacity check during request acceptance.

Q28 Skill demand versus mentor supply

Purpose: Compare how many students report interest in a skill and how many mentors list it.

DBMS concepts: [R/C] Pre-aggregated derived tables; avoid fan-out duplication.

```
SELECT
    s.skill_id,
    s.skill_name,
    COALESCE(sd.student_count, 0) AS students_with_skill,
    COALESCE(ms.mentor_count, 0) AS mentors_with_skill
FROM skill AS s
LEFT JOIN (
    SELECT skill_id, COUNT(DISTINCT student_id) AS student_count
    FROM student_skill
    GROUP BY skill_id
) AS sd
  ON sd.skill_id = s.skill_id
LEFT JOIN (
    SELECT skill_id, COUNT(DISTINCT mentor_id) AS mentor_count
    FROM mentor_skill
    GROUP BY skill_id
) AS ms
  ON ms.skill_id = s.skill_id
ORDER BY students_with_skill DESC,
         mentors_with_skill ASC,
         s.skill_name;
```

Implementation note: Aggregate each many-to-many table before joining, otherwise student × mentor combinations inflate counts.

9. Views and safe data presentation

Views provide reusable query interfaces. The blueprint specifically recommends a public mentor directory that does not reveal private fields. Views are also useful for dashboard summaries so the application does not duplicate complex SQL in several routes.

V01 - Public mentor directory

This view exposes discoverable professional fields only. It does not include email, phone, password hash or private account attributes.

```
CREATE OR REPLACE VIEW v_public_mentor_directory AS
SELECT
    mp.mentor_id,
    mp.full_name,
    mp.job_title,
    mp.current_city,
    mp.years_experience,
```

```
    mp.biography,
    c.company_name,
    c.industry,
    d.department_name
FROM mentor_profile AS mp
JOIN user_account AS ua
  ON ua.user_id = mp.user_id
LEFT JOIN company AS c
  ON c.company_id = mp.company_id
LEFT JOIN department AS d
  ON d.department_id = mp.department_id
WHERE ua.account_status = 'ACTIVE'
  AND mp.mentor_status = 'AVAILABLE';
```

V02 - Mentor review summary

```
CREATE OR REPLACE VIEW v_mentor_rating_summary AS
SELECT
    mp.mentor_id,
    mp.full_name,
    COUNT(mr.review_id) AS review_count,
    ROUND(AVG(mr.rating), 2) AS average_rating,
    MAX(mr.created_at) AS latest_review_at
FROM mentor_profile AS mp
LEFT JOIN mentor_review AS mr
  ON mr.mentor_id = mp.mentor_id
GROUP BY mp.mentor_id, mp.full_name;
```

V03 - Student request and allocation summary

Requests and allocations are aggregated independently before joining. This avoids request × allocation fan-out from inflating either count.

```
CREATE OR REPLACE VIEW v_student_mentorship_summary AS
SELECT
    sp.student_id,
    sp.full_name AS student_name,
    COALESCE(r.request_count, 0) AS request_count,
    COALESCE(r.pending_request_count, 0) AS pending_request_count,
    COALESCE(a.allocation_count, 0) AS allocation_count,
    COALESCE(a.active_allocation_count, 0) AS active_allocation_count
FROM student_profile AS sp
LEFT JOIN (
    SELECT student_id,
           COUNT(*) AS request_count,
           SUM(status = 'PENDING') AS pending_request_count
    FROM mentorship_request
    GROUP BY student_id
) AS r ON r.student_id = sp.student_id
LEFT JOIN (
    SELECT student_id,
           COUNT(*) AS allocation_count,
           SUM(status = 'ACTIVE') AS active_allocation_count
    FROM mentor_allocation
    GROUP BY student_id
) AS a ON a.student_id = sp.student_id;
```

V04 - Session outcome summary by mentor

```
CREATE OR REPLACE VIEW v_mentor_session_outcomes AS
SELECT
    mp.mentor_id,
    mp.full_name,
    COUNT(ms.session_id) AS total_sessions,
    SUM(ms.status = 'COMPLETED') AS completed_sessions,
    SUM(ms.status = 'CANCELLED') AS cancelled_sessions,
    SUM(ms.status = 'NO_SHOW') AS no_show_sessions
FROM mentor_profile AS mp
LEFT JOIN mentor_allocation AS ma
  ON ma.mentor_id = mp.mentor_id
LEFT JOIN mentorship_session AS ms
  ON ms.allocation_id = ma.allocation_id
GROUP BY mp.mentor_id, mp.full_name;
```

Querying a view

```
SELECT mentor_id, full_name, company_name, industry, years_experience
FROM v_public_mentor_directory
WHERE industry = @industry
ORDER BY years_experience DESC;
```

View security A view is not a substitute for API authorization. Grant application database users only the required privileges; keep administrative tables and private columns outside public-facing views.

10. Stored functions

A stored function returns one value and can be used inside a SELECT or other SQL expression. It is suitable for a small reusable derived value. Keep complex, multi-row workflows in stored procedures instead.

F01 - Convert a numeric rating to a label

Thresholds are proposed reporting labels, not source-defined platform policy. Keep them in configuration if administrators need to change them.

```
DELIMITER $$
CREATE FUNCTION fn_rating_label(p_rating DECIMAL(3,2))
RETURNS VARCHAR(20)
DETERMINISTIC
NO SQL
BEGIN
    RETURN CASE
        WHEN p_rating IS NULL THEN 'NOT_RATED'
        WHEN p_rating >= 4.50 THEN 'EXCELLENT'
        WHEN p_rating >= 4.00 THEN 'VERY_GOOD'
        WHEN p_rating >= 3.00 THEN 'GOOD'
        WHEN p_rating >= 2.00 THEN 'NEEDS_REVIEW'
        ELSE 'LOW_RATING'
    END;
END$$
DELIMITER ;

SELECT
    mentor_id,
    full_name,
    average_rating,
    fn_rating_label(average_rating) AS rating_label
FROM v_mentor_rating_summary;
```

Function versus procedure

Stored function	Stored procedure
Returns a scalar value via RETURN.	Performs a named database operation; can return result sets and OUT parameters.
Useful for labels, reusable calculations and expressions.	Useful for request acceptance, booking, cancellation and session updates.
Should remain deterministic / read-only where possible.	May own a transaction and coordinate multiple INSERT / UPDATE statements.
Can be used in SELECT expressions.	Invoked with CALL procedure_name(...).

11. Stored procedures and transactional operations

The project has operations that change multiple related rows. These should be centralised in stored procedures or a single application transaction with an equivalent tested boundary. The following MySQL procedure templates demonstrate the expected transaction pattern; field names must match the schema contract in Section 3. Each mutating procedure includes rollback/resignal handling, and the lookup routines convert a missing target row into a controlled SQL error.

Procedure design pattern Validate actor and state → lock authoritative rows → re-check capacity / conflicts → make all related writes → insert outbox event → COMMIT. On any SQL exception, ROLLBACK and RESIGNAL.

SP01 - Accept a mentorship request and create an allocation

The procedure locks the request and mentor row, checks that the request is still pending, validates mentor identity and account status, enforces active-mentee capacity and creates the allocation plus outbox event atomically.

This procedure uses the authenticated actor ID to verify that the caller owns the requested mentor profile; the API must also enforce the MENTOR role before calling it. A UNIQUE constraint on mentor_allocation.request_id remains the final duplicate-request safeguard.

```
DELIMITER $$
CREATE PROCEDURE sp_accept_mentorship_request(
    IN p_request_id BIGINT,
    IN p_mentor_id BIGINT,
    IN p_actor_user_id BIGINT
)
BEGIN
    DECLARE v_student_id BIGINT;
    DECLARE v_request_mentor_id BIGINT;
    DECLARE v_request_status VARCHAR(20);
    DECLARE v_mentor_user_id BIGINT;
    DECLARE v_mentor_status VARCHAR(20);
    DECLARE v_account_status VARCHAR(20);
    DECLARE v_capacity INT;
    DECLARE v_active_count INT DEFAULT 0;
    DECLARE v_duplicate_count INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR NOT FOUND
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Request, mentor profile or account record was not found';
    END;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT student_id, mentor_id, status
      INTO v_student_id, v_request_mentor_id, v_request_status
    FROM mentorship_request
    WHERE request_id = p_request_id
    FOR UPDATE;

    IF v_request_mentor_id IS NULL
       OR v_request_mentor_id <> p_mentor_id
       OR v_request_status <> 'PENDING' THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Request missing, not assigned to this mentor, or not pending';
    END IF;

    SELECT user_id, mentor_status, capacity
      INTO v_mentor_user_id, v_mentor_status, v_capacity
    FROM mentor_profile
    WHERE mentor_id = p_mentor_id
    FOR UPDATE;

    SELECT account_status
      INTO v_account_status
    FROM user_account
    WHERE user_id = v_mentor_user_id;

    IF v_mentor_user_id <> p_actor_user_id THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Only the assigned mentor may accept this request';
    END IF;

    IF v_mentor_status <> 'AVAILABLE'
       OR v_account_status <> 'ACTIVE'
       OR v_capacity IS NULL
       OR v_capacity < 0 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Mentor account/profile or capacity is not eligible';
    END IF;

    SELECT COUNT(*)
      INTO v_active_count
    FROM mentor_allocation
```

```
    WHERE mentor_id = p_mentor_id
      AND status = 'ACTIVE';

    IF v_active_count >= v_capacity THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Mentor has reached active mentee capacity';
    END IF;

    SELECT COUNT(*)
      INTO v_duplicate_count
    FROM mentor_allocation
    WHERE student_id = v_student_id
      AND mentor_id = p_mentor_id
      AND status = 'ACTIVE';

    IF v_duplicate_count > 0 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'An active allocation already exists for this student and mentor';
    END IF;

    INSERT INTO mentor_allocation
        (request_id, student_id, mentor_id, status,
         allocation_reason, created_at, start_date)
    VALUES
        (p_request_id, v_student_id, p_mentor_id, 'ACTIVE',
         'Accepted mentorship request', CURRENT_TIMESTAMP, CURRENT_DATE);

    UPDATE mentorship_request
    SET status = 'ACCEPTED',
        decision_at = CURRENT_TIMESTAMP
    WHERE request_id = p_request_id
      AND status = 'PENDING';

    INSERT INTO notification_outbox
        (event_type, aggregate_type, aggregate_id, payload,
         delivery_status, created_at)
    VALUES
        ('MENTORSHIP_REQUEST_ACCEPTED', 'MENTORSHIP_REQUEST', p_request_id,
         JSON_OBJECT('request_id', p_request_id,
                     'student_id', v_student_id,
                     'mentor_id', p_mentor_id,
                     'actor_user_id', p_actor_user_id),
         'PENDING', CURRENT_TIMESTAMP);

    COMMIT;
END$$
DELIMITER ;

CALL sp_accept_mentorship_request(501, 44, 9001);
```

SP02 - Create an atomic booking

The procedure uses MySQL as the final authority. It locks the student, availability and mentor rows; requires an active allocation under the current ER-aligned default; verifies mentor/account eligibility and capacity consistency; rejects time overlaps for either participant; then commits the booking, its scheduled session, the slot update and outbox event together. Existing allocated mentees are not rejected merely because the mentor is at the configured capacity limit.

A generated active_slot_key / unique index is recommended as a second line of defence. This procedure checks both student-side and mentor-side overlaps, not just the slot row. The current ER model makes allocation the parent of each actual session; one-off bookings without an allocation require an explicit ER/schema change before this procedure is relaxed.

```
DELIMITER $$
CREATE PROCEDURE sp_create_booking(
    IN p_actor_user_id BIGINT,
    IN p_availability_id BIGINT,
    IN p_booking_type VARCHAR(30),
    IN p_allocation_id BIGINT
)
BEGIN
    DECLARE v_student_id BIGINT;
    DECLARE v_mentor_id BIGINT;
    DECLARE v_start_at DATETIME;
    DECLARE v_end_at DATETIME;
    DECLARE v_slot_status VARCHAR(20);
    DECLARE v_mentor_status VARCHAR(20);
    DECLARE v_mentor_user_id BIGINT;
    DECLARE v_mentor_account_status VARCHAR(20);
    DECLARE v_capacity INT;
```

```
DECLARE v_active_mentees INT DEFAULT 0;
DECLARE v_student_conflicts INT DEFAULT 0;
DECLARE v_mentor_conflicts INT DEFAULT 0;
DECLARE v_allocation_count INT DEFAULT 0;
DECLARE v_booking_id BIGINT;

DECLARE EXIT HANDLER FOR NOT FOUND
BEGIN
    ROLLBACK;
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Student, availability, mentor or account record was not found';
END;

DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    RESIGNAL;
END;

START TRANSACTION;

SELECT sp.student_id
  INTO v_student_id
FROM student_profile AS sp
JOIN user_account AS ua ON ua.user_id = sp.user_id
WHERE ua.user_id = p_actor_user_id
  AND ua.account_status = 'ACTIVE'
FOR UPDATE;

SELECT mentor_id, start_at, end_at, slot_status
  INTO v_mentor_id, v_start_at, v_end_at, v_slot_status
FROM mentor_availability
WHERE availability_id = p_availability_id
FOR UPDATE;

IF v_slot_status <> 'AVAILABLE'
   OR v_start_at <= CURRENT_TIMESTAMP
   OR v_end_at <= v_start_at THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Availability slot is not currently bookable';
END IF;

SELECT mentor_status, user_id, capacity
  INTO v_mentor_status, v_mentor_user_id, v_capacity
FROM mentor_profile
WHERE mentor_id = v_mentor_id
FOR UPDATE;

SELECT account_status
  INTO v_mentor_account_status
FROM user_account
WHERE user_id = v_mentor_user_id;

IF v_mentor_status <> 'AVAILABLE'
   OR v_mentor_account_status <> 'ACTIVE'
   OR v_capacity IS NULL
   OR v_capacity < 0 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Mentor profile, account or capacity is not eligible for bookings';
END IF;

IF p_booking_type IS NULL OR p_booking_type NOT IN ('MENTORSHIP', 'MOCK_INTERVIEW') THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Unsupported booking type';
END IF;

/* Current conceptual ER baseline: every actual session belongs to an allocation.
   Unallocated one-off mock interviews require a deliberate ER/schema extension first. */
IF p_allocation_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'An active allocation is required for a booking in the current ER model';
END IF;

SELECT COUNT(*)
  INTO v_allocation_count
FROM mentor_allocation
WHERE allocation_id = p_allocation_id
  AND student_id = v_student_id
  AND mentor_id = v_mentor_id
  AND status = 'ACTIVE';

IF v_allocation_count <> 1 THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Allocation does not match student/mentor or is not active';
```

```
    END IF;

    SELECT COUNT(*)
      INTO v_active_mentees
    FROM mentor_allocation
    WHERE mentor_id = v_mentor_id
      AND status = 'ACTIVE';

    IF v_active_mentees > v_capacity THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Mentor has exceeded configured active-mentee capacity';
    END IF;

    SELECT COUNT(*)
      INTO v_student_conflicts
    FROM booking
    WHERE student_id = v_student_id
      AND status = 'CONFIRMED'
      AND start_at < v_end_at
      AND end_at > v_start_at;

    IF v_student_conflicts > 0 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Student already has an overlapping confirmed booking';
    END IF;

    SELECT COUNT(*)
      INTO v_mentor_conflicts
    FROM booking
    WHERE mentor_id = v_mentor_id
      AND status = 'CONFIRMED'
      AND start_at < v_end_at
      AND end_at > v_start_at;

    IF v_mentor_conflicts > 0 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Mentor already has an overlapping confirmed booking';
    END IF;

    INSERT INTO booking
        (availability_id, student_id, mentor_id, allocation_id,
         booking_type, start_at, end_at, status, created_at)
    VALUES
        (p_availability_id, v_student_id, v_mentor_id, p_allocation_id,
         p_booking_type, v_start_at, v_end_at, 'CONFIRMED', CURRENT_TIMESTAMP);

    SET v_booking_id = LAST_INSERT_ID();

    /* A confirmed booking materialises as one scheduled session under the current ER model. */
    INSERT INTO mentorship_session
        (booking_id, allocation_id, status, session_at, created_at)
    VALUES
        (v_booking_id, p_allocation_id, 'SCHEDULED', v_start_at, CURRENT_TIMESTAMP);

    UPDATE mentor_availability
    SET slot_status = 'BOOKED'
    WHERE availability_id = p_availability_id
      AND slot_status = 'AVAILABLE';

    IF ROW_COUNT() <> 1 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Availability slot was claimed by another booking';
    END IF;

    INSERT INTO notification_outbox
        (event_type, aggregate_type, aggregate_id, payload,
         delivery_status, created_at)
    VALUES
        ('BOOKING_CONFIRMED', 'BOOKING', v_booking_id,
         JSON_OBJECT('student_id', v_student_id,
                     'mentor_id', v_mentor_id,
                     'availability_id', p_availability_id),
         'PENDING', CURRENT_TIMESTAMP);

    COMMIT;
END$$
DELIMITER ;

CALL sp_create_booking(1001, 880, 'MENTORSHIP', 300);
```

SP03 - Cancel a booking and retain history

If administrators may cancel bookings, extend the authorization predicate with an administrator-role check enforced by the backend / database security model. The scheduled session is marked CANCELLED but retained for history; the future slot is reopened. Apply the configured cancellation cutoff if the team adopts one.

```
DELIMITER $$
CREATE PROCEDURE sp_cancel_booking(
    IN p_booking_id BIGINT,
    IN p_actor_user_id BIGINT,
    IN p_reason VARCHAR(500)
)
BEGIN
    DECLARE v_student_id BIGINT;
    DECLARE v_mentor_id BIGINT;
    DECLARE v_availability_id BIGINT;
    DECLARE v_status VARCHAR(20);
    DECLARE v_student_user_id BIGINT;
    DECLARE v_mentor_user_id BIGINT;
    DECLARE v_authorized INT DEFAULT 0;

    DECLARE EXIT HANDLER FOR NOT FOUND
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Booking or participant profile was not found';
    END;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT student_id, mentor_id, availability_id, status
      INTO v_student_id, v_mentor_id, v_availability_id, v_status
    FROM booking
    WHERE booking_id = p_booking_id
    FOR UPDATE;

    SELECT user_id INTO v_student_user_id
    FROM student_profile WHERE student_id = v_student_id;
    SELECT user_id INTO v_mentor_user_id
    FROM mentor_profile WHERE mentor_id = v_mentor_id;

    IF p_actor_user_id = v_student_user_id
       OR p_actor_user_id = v_mentor_user_id THEN
        SET v_authorized = 1;
    END IF;

    IF v_authorized = 0 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Actor is not a participant in this booking';
    END IF;

    IF v_status <> 'CONFIRMED' THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Only confirmed bookings can be cancelled';
    END IF;

    UPDATE booking
    SET status = 'CANCELLED',
        cancelled_at = CURRENT_TIMESTAMP
    WHERE booking_id = p_booking_id;

    UPDATE mentorship_session
    SET status = 'CANCELLED'
    WHERE booking_id = p_booking_id
      AND status = 'SCHEDULED';

    UPDATE mentor_availability
    SET slot_status = CASE
        WHEN start_at > CURRENT_TIMESTAMP THEN 'AVAILABLE'
        ELSE slot_status
    END
    WHERE availability_id = v_availability_id;

    INSERT INTO notification_outbox
        (event_type, aggregate_type, aggregate_id, payload,
         delivery_status, created_at)
    VALUES
```

```
        ('BOOKING_CANCELLED', 'BOOKING', p_booking_id,
         JSON_OBJECT('actor_user_id', p_actor_user_id,
                     'reason', p_reason,
                     'student_id', v_student_id,
                     'mentor_id', v_mentor_id),
         'PENDING', CURRENT_TIMESTAMP);

    COMMIT;
END$$
DELIMITER ;
```

SP04 - Record the actual session outcome

The supplied workflow defines Scheduled → Completed / Cancelled / No-show. The application should keep session history and make the outcome change auditable.

```
DELIMITER $$
CREATE PROCEDURE sp_record_session_outcome(
    IN p_session_id BIGINT,
    IN p_actor_user_id BIGINT,
    IN p_new_status VARCHAR(20),
    IN p_topics TEXT,
    IN p_skills_assessed TEXT,
    IN p_observations TEXT,
    IN p_improvement_areas TEXT,
    IN p_next_steps TEXT,
    IN p_recommended_preparation TEXT
)
BEGIN
    DECLARE v_mentor_user_id BIGINT;
    DECLARE v_old_status VARCHAR(20);
    DECLARE v_booking_id BIGINT;

    DECLARE EXIT HANDLER FOR NOT FOUND
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Session, booking or mentor profile was not found';
    END;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT ms.status, ms.booking_id, mp.user_id
      INTO v_old_status, v_booking_id, v_mentor_user_id
    FROM mentorship_session AS ms
    JOIN booking AS b ON b.booking_id = ms.booking_id
    JOIN mentor_profile AS mp ON mp.mentor_id = b.mentor_id
    WHERE ms.session_id = p_session_id
    FOR UPDATE;

    IF p_actor_user_id <> v_mentor_user_id THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Only the assigned mentor may record this session outcome';
    END IF;

    IF v_old_status <> 'SCHEDULED' THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Only scheduled sessions can receive an outcome';
    END IF;

    IF p_new_status NOT IN ('COMPLETED','CANCELLED','NO_SHOW') THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Invalid session outcome';
    END IF;

    UPDATE mentorship_session
    SET status = p_new_status,
        topics = p_topics,
        skills_assessed = p_skills_assessed,
        observations = p_observations,
        improvement_areas = p_improvement_areas,
        next_steps = p_next_steps,
        recommended_preparation = p_recommended_preparation,
        completed_at = CASE WHEN p_new_status = 'COMPLETED'
                            THEN CURRENT_TIMESTAMP ELSE completed_at END
    WHERE session_id = p_session_id;
```

```
    UPDATE booking
    SET status = CASE
        WHEN p_new_status = 'COMPLETED' THEN 'COMPLETED'
        WHEN p_new_status = 'NO_SHOW' THEN 'NO_SHOW'
        ELSE 'CANCELLED'
    END
    WHERE booking_id = v_booking_id;

    INSERT INTO notification_outbox
        (event_type, aggregate_type, aggregate_id, payload,
         delivery_status, created_at)
    VALUES
        ('SESSION_OUTCOME_RECORDED', 'MENTORSHIP_SESSION', p_session_id,
         JSON_OBJECT('status', p_new_status,
                     'actor_user_id', p_actor_user_id),
         'PENDING', CURRENT_TIMESTAMP);

    COMMIT;
END$$
DELIMITER ;
```

SP05 - Submit a mentor review

The procedure resolves the student profile from p_actor_user_id rather than trusting a client-supplied student_id. Add UNIQUE(session_id, student_id) to mentor_review; the trigger in Section 12 provides an additional direct-INSERT guard.

```
DELIMITER $$
CREATE PROCEDURE sp_submit_mentor_review(
    IN p_session_id BIGINT,
    IN p_actor_user_id BIGINT,
    IN p_rating TINYINT,
    IN p_review_text TEXT
)
BEGIN
    DECLARE v_session_status VARCHAR(20);
    DECLARE v_booking_student_id BIGINT;
    DECLARE v_mentor_id BIGINT;
    DECLARE v_actor_student_id BIGINT;

    DECLARE EXIT HANDLER FOR NOT FOUND
    BEGIN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Session, booking or student profile was not found';
    END;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    START TRANSACTION;

    SELECT ms.status, b.student_id, b.mentor_id
      INTO v_session_status, v_booking_student_id, v_mentor_id
    FROM mentorship_session AS ms
    JOIN booking AS b ON b.booking_id = ms.booking_id
    WHERE ms.session_id = p_session_id
    FOR UPDATE;

    SELECT student_id
      INTO v_actor_student_id
    FROM student_profile
    WHERE user_id = p_actor_user_id;

    IF v_session_status <> 'COMPLETED'
       OR v_booking_student_id <> v_actor_student_id THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Review requires a completed session owned by the authenticated student';
    END IF;

    IF p_rating IS NULL OR p_rating < 1 OR p_rating > 5 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Rating must be between 1 and 5';
    END IF;

    INSERT INTO mentor_review
        (session_id, student_id, mentor_id, rating, review_text, created_at)
    VALUES
        (p_session_id, p_student_id, v_mentor_id,
         p_rating, p_review_text, CURRENT_TIMESTAMP);
```

```
    INSERT INTO notification_outbox
        (event_type, aggregate_type, aggregate_id, payload,
         delivery_status, created_at)
    VALUES
        ('MENTOR_REVIEW_SUBMITTED', 'MENTOR_REVIEW', LAST_INSERT_ID(),
         JSON_OBJECT('session_id', p_session_id,
                     'student_id', v_actor_student_id,
                     'mentor_id', v_mentor_id),
         'PENDING', CURRENT_TIMESTAMP);

    COMMIT;
END$$
DELIMITER ;
```

11.1 Recommended uniqueness and range constraints

The generated key preserves cancelled booking history while preventing two non-cancelled bookings from claiming the same slot. Review generated-column syntax and existing indexes against the deployed schema before applying.

```
-- Apply only after checking that equivalent constraints do not already exist.
ALTER TABLE mentor_allocation
  ADD CONSTRAINT uq_allocation_request UNIQUE (request_id);

ALTER TABLE mentor_review
  ADD CONSTRAINT uq_review_session_student UNIQUE (session_id, student_id),
  ADD CONSTRAINT chk_mentor_review_rating CHECK (rating BETWEEN 1 AND 5);

ALTER TABLE recommended_mentor
  ADD CONSTRAINT uq_recommendation_run_mentor UNIQUE (run_id, mentor_id),
  ADD CONSTRAINT chk_recommendation_score CHECK (score BETWEEN 0 AND 100);

ALTER TABLE mentorship_session
  ADD CONSTRAINT uq_session_booking UNIQUE (booking_id);

-- Optional second line of defence for one active reservation per availability slot.
ALTER TABLE booking
  ADD COLUMN active_slot_key BIGINT
    GENERATED ALWAYS AS (
      CASE WHEN status IN ('CONFIRMED','COMPLETED','NO_SHOW')
           THEN availability_id ELSE NULL END
    ) STORED;

CREATE UNIQUE INDEX uq_booking_active_slot
  ON booking(active_slot_key);
```

12. Triggers and audit enforcement

A trigger runs automatically when an INSERT, UPDATE or DELETE event occurs. The supplied Unit 2 material covers BEFORE / AFTER triggers, row-level execution, NEW and OLD values, and SIGNAL SQLSTATE for rejecting invalid data. In AlumniConnect, triggers are best used for local validation and audit history. Use stored procedures / application transactions for cross-table workflows.

Do not send external notifications from a trigger Write a durable outbox row inside the same transaction; a worker delivers the notification after commit. Avoid duplicating the same outbox event in both a trigger and a stored procedure.

TR01 - Audit request status changes

The <=> operator is MySQL NULL-safe equality. The application should set @app_actor_user_id on the connection before a write and clear it before returning a pooled connection.

```
DELIMITER $$
CREATE TRIGGER trg_request_status_audit
AFTER UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
    IF NOT (OLD.status <=> NEW.status) THEN
        INSERT INTO audit_log
            (entity_type, record_id, action, actor_user_id,
             old_values, new_values, occurred_at)
        VALUES
            ('MENTORSHIP_REQUEST', OLD.request_id, 'STATUS_CHANGED',
             @app_actor_user_id,
```

```
            JSON_OBJECT('status', OLD.status),
            JSON_OBJECT('status', NEW.status),
            CURRENT_TIMESTAMP);
    END IF;
END$$
DELIMITER ;
```

TR02 - Reject invalid request-status transitions

This encodes the canonical request lifecycle. If the team later permits an audited administrative correction, implement it as a separately authorized correction routine rather than silently weakening all transitions.

```
DELIMITER $$
CREATE TRIGGER trg_request_status_guard
BEFORE UPDATE ON mentorship_request
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        IF OLD.status <> 'PENDING'
           OR NEW.status NOT IN ('ACCEPTED','REJECTED','CANCELLED') THEN
            SIGNAL SQLSTATE '45000'
              SET MESSAGE_TEXT = 'Invalid mentorship request status transition';
        END IF;
    END IF;
END$$
DELIMITER ;
```

TR03 - Validate session status transitions

```
DELIMITER $$
CREATE TRIGGER trg_session_status_guard
BEFORE UPDATE ON mentorship_session
FOR EACH ROW
BEGIN
    IF OLD.status <> NEW.status THEN
        IF OLD.status <> 'SCHEDULED'
           OR NEW.status NOT IN ('COMPLETED','CANCELLED','NO_SHOW') THEN
            SIGNAL SQLSTATE '45000'
              SET MESSAGE_TEXT = 'Invalid mentorship session status transition';
        END IF;
    END IF;

    IF NEW.status = 'COMPLETED' AND NEW.completed_at IS NULL THEN
        SET NEW.completed_at = CURRENT_TIMESTAMP;
    END IF;
END$$
DELIMITER ;
```

TR04 - Reject mentor reviews for invalid sessions

Add a UNIQUE(session_id, student_id) constraint too. A trigger validates cross-table rules; a unique key protects the one-review-per-session invariant under concurrency.

```
DELIMITER $$
CREATE TRIGGER trg_mentor_review_validate
BEFORE INSERT ON mentor_review
FOR EACH ROW
BEGIN
    DECLARE v_session_status VARCHAR(20);
    DECLARE v_student_id BIGINT;
    DECLARE v_mentor_id BIGINT;

    SELECT ms.status, b.student_id, b.mentor_id
      INTO v_session_status, v_student_id, v_mentor_id
    FROM mentorship_session AS ms
    JOIN booking AS b ON b.booking_id = ms.booking_id
    WHERE ms.session_id = NEW.session_id;

    IF v_session_status IS NULL
       OR v_session_status <> 'COMPLETED'
       OR NEW.student_id <> v_student_id
       OR NEW.mentor_id <> v_mentor_id THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Review must reference a completed session and its student/mentor';
    END IF;

    IF NEW.rating IS NULL OR NEW.rating < 1 OR NEW.rating > 5 THEN
        SIGNAL SQLSTATE '45000'
          SET MESSAGE_TEXT = 'Mentor review rating must be between 1 and 5';
    END IF;
END$$
DELIMITER ;
```

12.1 Trigger coverage matrix

Trigger	Event	Responsibility	Why trigger / separate procedure
trg_request_status_guard	BEFORE UPDATE request	Reject invalid request state transitions.	Small invariant on one row.
trg_request_status_audit	AFTER UPDATE request	Record old/new request state.	Automatically capture every direct update path.
trg_session_status_guard	BEFORE UPDATE session	Restrict lifecycle and set completed_at.	Small invariant on one row.
trg_mentor_review_validate	BEFORE INSERT review	Ensure completed session / correct student and mentor.	Cross-table guard on review insertion; unique key still required.
sp_accept_mentorship_request	CALL procedure	Lock, enforce capacity, create allocation + outbox.	Multi-row transactional operation.
sp_create_booking	CALL procedure	Lock slot, validate overlaps, create booking + outbox.	Concurrency-sensitive operation.

13. CTEs, window functions and full-text search

The project blueprint explicitly calls out a CTE-based request-to-allocation-to-session funnel, window functions to rank mentors within a domain and full-text search across alumni experience and biographies. These examples demonstrate advanced Unit 2 SQL in a meaningful platform context.

Q29 Request-to-allocation-to-session funnel using CTEs

Purpose: Measure request cohorts through acceptance, allocation and completed sessions.

DBMS concepts: [B/C] Common Table Expressions, aggregate counts, conditional aggregation.

```
WITH request_cohort AS (
    SELECT
        mr.request_id,
        mr.student_id,
        mr.status,
        mr.submitted_at,
        DATE_FORMAT(mr.submitted_at, '%Y-%m') AS cohort_month
    FROM mentorship_request AS mr
    WHERE mr.submitted_at >= @period_start
      AND mr.submitted_at <  @period_end
), allocation_by_request AS (
    SELECT
        ma.request_id,
        ma.allocation_id
    FROM mentor_allocation AS ma
), sessions_by_allocation AS (
    SELECT
        ms.allocation_id,
        COUNT(*) AS completed_sessions
    FROM mentorship_session AS ms
    WHERE ms.status = 'COMPLETED'
    GROUP BY ms.allocation_id
)
SELECT
    rc.cohort_month,
    COUNT(DISTINCT rc.request_id) AS requests_submitted,
    COUNT(DISTINCT CASE WHEN rc.status = 'ACCEPTED'
                        THEN rc.request_id END) AS requests_accepted,
    COUNT(DISTINCT abr.allocation_id) AS allocations_created,
    COALESCE(SUM(sba.completed_sessions), 0) AS completed_sessions
FROM request_cohort AS rc
LEFT JOIN allocation_by_request AS abr
  ON abr.request_id = rc.request_id
LEFT JOIN sessions_by_allocation AS sba
  ON sba.allocation_id = abr.allocation_id
GROUP BY rc.cohort_month
ORDER BY rc.cohort_month;
```

Implementation note: This is a cohort-style funnel based on the request submission month. If you need acceptance-month analytics, persist / audit decision_at and group by the decision timestamp instead.

Q30 Rank mentors within each mentorship domain

Purpose: Rank mentors by average student review rating within each expertise domain.

DBMS concepts: [B/C] CTE + window function RANK() OVER(PARTITION BY ...).

```
WITH mentor_ratings AS (
    SELECT
        mr.mentor_id,
        COUNT(*) AS review_count,
        AVG(mr.rating) AS average_rating
    FROM mentor_review AS mr
    GROUP BY mr.mentor_id
), domain_scores AS (
    SELECT
        md.domain_id,
        md.domain_name,
        mp.mentor_id,
        mp.full_name,
        COALESCE(mr.review_count, 0) AS review_count,
        mr.average_rating,
        mp.years_experience
    FROM mentor_domain_expertise AS mde
    JOIN mentorship_domain AS md
      ON md.domain_id = mde.domain_id
    JOIN mentor_profile AS mp
      ON mp.mentor_id = mde.mentor_id
    LEFT JOIN mentor_ratings AS mr
      ON mr.mentor_id = mp.mentor_id
)
SELECT
    domain_name,
    mentor_id,
    full_name,
    review_count,
    ROUND(average_rating, 2) AS average_rating,
    years_experience,
    RANK() OVER (
        PARTITION BY domain_id
        ORDER BY average_rating DESC,
                 years_experience DESC
    ) AS domain_rank
FROM domain_scores
WHERE review_count >= 3
ORDER BY domain_name, domain_rank;
```

Implementation note: RANK() gives tied values the same rank. Use ROW_NUMBER() where a unique deterministic ordinal is required. The review-count filter avoids ranking mentors on a single review.

Q31 Latest session per allocation (ROW_NUMBER)

Purpose: Retrieve the latest recorded session for every mentorship allocation.

DBMS concepts: [C] Window function ROW_NUMBER() OVER(PARTITION BY ...).

```
WITH ranked_sessions AS (
    SELECT
        ms.*,
        ROW_NUMBER() OVER (
            PARTITION BY ms.allocation_id
            ORDER BY ms.session_at DESC, ms.session_id DESC
        ) AS row_num
    FROM mentorship_session AS ms
)
SELECT
    allocation_id,
    session_id,
    session_at,
    status,
    topics,
    next_steps
FROM ranked_sessions
WHERE row_num = 1
ORDER BY session_at DESC;
```

Q32 - Full-text search in mentor biographies

The query below assumes a FULLTEXT index exists on biography, professional_summary and job_title. Use it for keyword-oriented discovery. The optional vector/semantic search feature is Phase 2 in the blueprint and is not a requirement for the core SQL release.

```
-- Create once; skip if an equivalent FULLTEXT index already exists.
```

```
ALTER TABLE mentor_profile
  ADD FULLTEXT INDEX ft_mentor_bio
      (biography, professional_summary, job_title);

SELECT
    mp.mentor_id,
    mp.full_name,
    mp.job_title,
    MATCH(mp.biography, mp.professional_summary, mp.job_title)
      AGAINST ('+machine +learning +infrastructure' IN BOOLEAN MODE)
      AS relevance_score
FROM mentor_profile AS mp
WHERE MATCH(mp.biography, mp.professional_summary, mp.job_title)
      AGAINST ('+machine +learning +infrastructure' IN BOOLEAN MODE)
  AND mp.mentor_status = 'AVAILABLE'
ORDER BY relevance_score DESC;
```

Use InnoDB FULLTEXT syntax supported by the target MySQL version. MySQL full-text search is token-based and language/configuration dependent; it is not semantic similarity. If the project adopts Redis Stack vector search later, combine it with Neo4j candidate retrieval and MySQL eligibility checks rather than making vector scores an authority.

14. Indexing and query optimisation

The Unit 3 material expects the team to show an initial query plan, add relevant indexes, rerun the query, compare plans and explain the performance difference. Index design should follow actual WHERE, JOIN, GROUP BY and ORDER BY patterns; do not add every possible index indiscriminately.

14.1 Recommended secondary indexes

Index names are recommendations. Check SHOW INDEX FROM <table> first and avoid duplicate indexes with the same leading columns.

```
-- Mentor discovery by skill / domain
CREATE INDEX ix_student_skill_skill_student
    ON student_skill(skill_id, student_id);
CREATE INDEX ix_mentor_skill_skill_mentor
    ON mentor_skill(skill_id, mentor_id);
CREATE INDEX ix_student_domain_domain_student
    ON student_domain_interest(domain_id, student_id);
CREATE INDEX ix_mentor_domain_domain_mentor
    ON mentor_domain_expertise(domain_id, mentor_id);

-- Request inbox and student history
CREATE INDEX ix_request_mentor_status_submitted
    ON mentorship_request(mentor_id, status, submitted_at);
CREATE INDEX ix_request_student_submitted
    ON mentorship_request(student_id, submitted_at);

-- Allocation load and student history
CREATE INDEX ix_allocation_mentor_status
    ON mentor_allocation(mentor_id, status);
CREATE INDEX ix_allocation_student_status
    ON mentor_allocation(student_id, status);

-- Schedule / overlap and session reports
CREATE INDEX ix_booking_student_status_start
    ON booking(student_id, status, start_at);
CREATE INDEX ix_booking_mentor_status_start
    ON booking(mentor_id, status, start_at);
CREATE INDEX ix_session_allocation_status_date
    ON mentorship_session(allocation_id, status, session_at);

-- Mentor review aggregate by mentor
CREATE INDEX ix_review_mentor_created
    ON mentor_review(mentor_id, created_at);

-- Outbox worker polling
CREATE INDEX ix_outbox_status_created
    ON notification_outbox(delivery_status, created_at);
```

14.2 EXPLAIN the mentor discovery query

```
EXPLAIN FORMAT=TREE
SELECT
    mp.mentor_id,
    mp.full_name,
    COUNT(DISTINCT ms.skill_id) AS matched_skill_count
FROM student_skill AS ss
```

```
JOIN mentor_skill AS ms
  ON ms.skill_id = ss.skill_id
JOIN mentor_profile AS mp
  ON mp.mentor_id = ms.mentor_id
WHERE ss.student_id = @student_id
GROUP BY mp.mentor_id, mp.full_name
ORDER BY matched_skill_count DESC;
```

Record the access path, chosen indexes, estimated rows and join order. Run EXPLAIN ANALYZE in a controlled test environment to observe actual rows and timing; it executes the SELECT rather than merely estimating it.

14.3 EXPLAIN ANALYZE example

```
EXPLAIN ANALYZE
SELECT
    mr.mentor_id,
    COUNT(*) AS pending_count
FROM mentorship_request AS mr
WHERE mr.status = 'PENDING'
  AND mr.submitted_at >= DATE_SUB(CURRENT_TIMESTAMP, INTERVAL 30 DAY)
GROUP BY mr.mentor_id;
```

14.4 Query-writing practices

- Filter on indexed raw columns where possible. Avoid wrapping an indexed timestamp in DATE() in the WHERE predicate when a timestamp range expresses the same requirement.
- Use DISTINCT only when duplicate elimination is part of the requirement; unnecessary DISTINCT can add sorting or temporary-table work.
- Aggregate child tables before joining several one-to-many relationships, or counts can be multiplied by join fan-out.
- Select only fields needed by the endpoint or report; avoid SELECT * in public APIs.
- Use keyset pagination for deep result sets where appropriate; OFFSET pagination becomes more expensive as offsets grow.
- Measure with representative data volumes. The earlier mini-project estimates 500–2,000 students, 1,000–5,000 alumni, thousands of skill associations and up to 10,000 sessions; scale synthetic data if needed for optimisation demonstrations.
- Do not use a cached result to make a booking decision. Re-read and validate current slot state inside the MySQL transaction.

15. Transactions, concurrency and outbox integration

15.1 Where transactions are required

Operation	Rows that commit together	Failure behaviour
Accept request	mentorship_request status; mentor_allocation row; notification_outbox event; audit_log entry (if transaction-scoped).	No allocation or outbox event is retained if validation or insert fails.
Create booking	booking row; mentor_availability slot status; notification_outbox event.	No partial reservation. Competing caller gets a conflict/unavailable result.
Cancel booking	booking status/history; availability slot recalculation; outbox event.	No cancellation notification if the cancellation transaction rolls back.
Record session outcome	mentorship_session status/outcome; related booking disposition; outbox event; audit.	No partial outcome. Preserve the existing scheduled state on rollback.
Submit mentor review	mentor_review row; outbox event.	No review if session is not completed or author is not the student participant.

15.2 Why locking matters for booking

A simple “SELECT available slot” followed by INSERT is unsafe when two students act at nearly the same time. The slot row must be locked and revalidated. Also check student-side overlaps and mentor-side overlaps. A unique constraint on the active slot key acts as a second line of defence, while row locks serialize the business operation.

```
-- Demonstration pattern inside a transaction (illustrative):
START TRANSACTION;

SELECT mentor_id, start_at, end_at, slot_status
FROM mentor_availability
WHERE availability_id = @availability_id
FOR UPDATE;

-- Re-check slot_status, student overlap, mentor overlap and allocation validity.
-- Only after all checks pass:
INSERT INTO booking (...)
VALUES (...);

UPDATE mentor_availability
SET slot_status = 'BOOKED'
WHERE availability_id = @availability_id
  AND slot_status = 'AVAILABLE';

COMMIT;

-- If any validation fails, issue ROLLBACK instead of a partial write.
```

15.3 Transactional outbox pattern

The request, booking, session and review routines insert an outbox event in the same MySQL transaction as the business write. A background worker reads committed outbox events, sends in-app/email notifications as configured, updates Neo4j projections and invalidates relevant Redis recommendation/profile cache keys. Use a stable event identifier and idempotent worker operations to make retries safe.

Outbox event	Typical recipient / consumer
MENTORSHIP_REQUEST_SUBMITTED	Assigned mentor.
MENTORSHIP_REQUEST_ACCEPTED / REJECTED	Student who submitted the request.
BOOKING_CONFIRMED / BOOKING_CANCELLED	Student and mentor participants.
SESSION_REMINDER	Both participants.
SESSION_OUTCOME_RECORDED	Relevant participants / analytics projection.
MENTOR_REVIEW_SUBMITTED	Relevant mentor / review summary projection.
MENTOR_PROFILE_UPDATED / SKILLS_UPDATED	Neo4j graph projection update and Redis cache invalidation.

Consistency rule If Redis or Neo4j disagrees with MySQL, MySQL wins. If the graph projection is delayed, recommendations may be stale; if Redis is unavailable, discovery can fall back to the database / graph path. Neither condition may create an invalid allocation or booking.

16. Administrator analytics catalogue

The PRD expects SQL-backed oversight for the request-to-allocation-to-session funnel, mentor ratings, completed sessions, mentors by domain and students without an allocation. The following dashboard catalogue connects each metric to a specific query or view.

Dashboard / metric	Definition	SQL source
Request funnel	Submitted, accepted, rejected and cancelled requests by submission cohort.	Q29 / request cohort CTE.
Allocation conversion	Accepted requests with an allocation ÷ submitted requests.	Q29 plus request/allocation join.
Completed sessions	Count of completed session records over selected period.	Q18 / Q25 / v_mentor_session_outcomes.
Mentor quality	Average mentor review rating and review count; require a minimum sample threshold for ranking.	Q22 / v_mentor_rating_summary.
Mentor load	Active allocation count against mentor capacity.	Q13 / Q27.
Domain demand	Request volume by mentorship domain.	Q21 / Q24.
Department rating	Average review rating by mentor department.	Q23.
No recent session	Active allocations without completed session in chosen period.	Q19.

Dashboard / metric	Definition	SQL source
Booking reliability	Cancellation and no-show percentage by month.	Q26.
Skill supply / demand	Students listing a skill compared with mentors listing that skill.	Q28.
Recommendation performance	Candidate rank/score, selection rate and later completed sessions by criteria_version.	recommendation_run + recommended_mentor + request/allocation/session joins.

16.1 Recommendation performance query

```
SELECT
    rr.criteria_version,
    COUNT(*) AS candidate_rows,
    SUM(rm.selected_flag = 1) AS selected_candidates,
    ROUND(100.0 * SUM(rm.selected_flag = 1) /
          NULLIF(COUNT(*), 0), 2) AS selection_pct
FROM recommendation_run AS rr
JOIN recommended_mentor AS rm
  ON rm.run_id = rr.run_id
GROUP BY rr.criteria_version
ORDER BY rr.criteria_version;
```

This measures selection from displayed candidate rows; it is not a causal measure of mentor quality. For eventual outcome tracking, join the selected candidate to the later mentorship_request / mentor_allocation / mentorship_session chain and define a time window.

16.2 Admin report privacy

- Prefer aggregate reports for platform oversight.
- Expose personally identifiable fields only when required for a specific authorised administrative workflow.
- Use row-level ownership checks for student and mentor portals even when data is queried through a view.
- Keep audit_log and notification_outbox access restricted to operational roles.

17. Verification and acceptance-test matrix

Use the following tests to demonstrate correctness. For concurrency tests, use two independent MySQL sessions and record the SQL statements, isolation level, result and final row counts.

Test ID	Scenario	Expected result	Related object
SQL-01	Mentor directory	Only active, available mentors are returned; private auth fields are absent.	Q02 / V01
SQL-02	Skill matching	A candidate with two overlapping skills ranks above a candidate with one, all else equal.	Q07
SQL-03	Domain matching	A candidate with no shared skill but matching preferred domain is discoverable.	Q08 / Q09
SQL-04	UNION de-duplication	Mentor returned through both skill and domain path appears once.	Q09
SQL-05	NULL handling	Missing company/job title is flagged; NULL is not compared with = NULL.	Q06
SQL-06	Correlated query	Student with no active allocation appears; student with active allocation does not.	Q10
SQL-07	Nested query	Experience comparison uses average within the same domain.	Q11
SQL-08	Expert skill filter	Only Expert-level match for selected skill is returned.	Q12
SQL-09	Mentor load	Only ACTIVE allocations count against capacity.	Q13 / Q27
SQL-10	Request inbox	Mentor sees only own PENDING requests.	Q15

Test ID	Scenario	Expected result	Related object
SQL-11	History retention	Rejected and cancelled request rows remain visible in history.	Q16
SQL-12	Session separation	Cancelled booking does not create a completed session.	Q17 / Q20
SQL-13	90-day activity	Student with no completed session in the last 90 days is flagged.	Q19
SQL-14	Review average	Mentor with no review has NULL average, not zero.	Q22 / Q23
SQL-15	Funnel counts	One request with multiple sessions is counted as one request and multiple sessions.	Q29
SQL-16	Window ranking	Mentors are ranked independently within each domain.	Q30
SQL-17	Full-text search	FULLTEXT index exists and query returns a relevance score.	Q32
SQL-18	Request guard	PENDING can transition to ACCEPTED/REJECTED/CANCELLED; terminal state cannot be changed normally.	TR02
SQL-19	Duplicate allocation	Same request cannot produce two allocation rows.	SP01 + UNIQUE(request_id)
SQL-20	Capacity race	Two concurrent acceptances cannot exceed mentor capacity.	SP01 + mentor row lock
SQL-21	Booking slot race	Two students contend for same availability row; at most one booking succeeds.	SP02 + slot lock + unique key
SQL-22	Student overlap	One student cannot hold overlapping CONFIRMED bookings with different mentors.	SP02
SQL-23	Mentor overlap	One mentor cannot hold overlapping CONFIRMED bookings with different slots.	SP02
SQL-24	Rollback	Force an error after booking insert; confirm booking, slot change and outbox row all roll back.	SP02 transaction
SQL-25	Booking cancellation	Cancellation preserves booking history and may re-open only a future slot.	SP03
SQL-26	Session transition	Only SCHEDULED session transitions to COMPLETED/CANCELLED/NO_SHOW.	TR03 / SP04
SQL-27	Review eligibility	Review before completion or by wrong student is rejected.	SP05 / TR04
SQL-28	One review	Second review for same session/student is blocked by UNIQUE constraint.	SP05 + unique key
SQL-29	Outbox rollback	Rolled-back business operation leaves no committed outbox event.	Transaction test
SQL-30	Worker retry	Reprocessing same event does not create duplicate side effects.	Outbox worker idempotency
SQL-31	Query plan	Record EXPLAIN / EXPLAIN ANALYZE before and after selected indexes.	Section 14
SQL-32	Cache safety	Redis cache hit never bypasses MySQL authorization or booking validation.	Architecture / PRD NFR-04

18. DBMS syllabus mapping and implementation order

DBMS unit	Course concepts	AlumniConnect SQL demonstration
Unit 1 - Introduction / relational foundations	ER model, relational schema, PK/FK, constraints, SELECT/WHERE, basic joins and relational operations.	Schema contract, profile retrieval, mentor directory, associative tables and referential-integrity checks (Q01-Q08).

DBMS unit	Course concepts	AlumniConnect SQL demonstration
Unit 2 - Advanced SQL	Set operations, NULL, aggregates, nested/correlated subqueries, joins, views, triggers, functions, procedures, CTEs, windows and full text.	Query catalogue Q09-Q32; V01-V04; F01; SP01-SP05; TR01-TR04.
Unit 3 - Advanced design and implementation	Functional dependencies, 1NF-3NF/BCNF, query processing and query optimisation.	Map query access paths to indexes, compare EXPLAIN plans, document candidate keys and normalization decisions in the relational design spec.
Unit 4 - Advanced databases	ACID, transactions, concurrency control, locking, Redis, Neo4j and optional vector search.	Atomic request acceptance and booking, lock tests, outbox, Redis result cache and Neo4j mentor discovery projection.

18.1 Suggested implementation sequence

1. Freeze physical names and keys using the Relational Design Specification. Confirm which old 12-table records are migrated and which new entities are created.
2. Create core DDL, PK/FK/UNIQUE/CHECK constraints and status domains. Load reference data for department, company, skill and mentorship domain.
3. Implement read queries Q01-Q20 and the public / admin views. Verify role and ownership filtering in FastAPI.
4. Implement aggregate / analytics queries Q21-Q31 and full-text index Q32. Add indexes only after reviewing EXPLAIN plans.
5. Implement SP01-SP05 and TR01-TR04 in a test schema. Test rollback, slot conflicts, duplicate allocation, session outcomes and review eligibility.
6. Implement notification_outbox worker and Redis invalidation. Add Neo4j projection update from committed profile/skill/domain events.
7. Run the full acceptance matrix with synthetic data at the target scale; store EXPLAIN output, latency and final row counts in the project report.

Appendix A. Source-document traceability

The catalogue is grounded in the PDFs and project files supplied earlier in this conversation. Where a companion document defines a conceptual entity or requirement but does not prescribe a physical column name, the SQL contract in Section 3 provides an explicit implementation assumption.

Supplied source	Relevant content used in this catalogue
Alumni–Student Mentorship Platform: DBMS Project Blueprint (34-page version)	Project scope and roles; modular monolith; MySQL / Redis / Neo4j responsibilities; existing mini-project; 12 core entities; workflow design; Unit 2 SQL showcase; query optimisation and Unit 4 database demonstrations.
AlumniConnect Application Architecture (20-page design)	Module boundaries; MySQL entity domains and cardinalities; Redis cache-aside behaviour; Neo4j relationship projection; outbox; mentor discovery path; SQL syllabus mapping.
AlumniConnect Conceptual ER Diagram & Domain Model (18 pages)	Entity catalogue and relationship catalogue: Recommendation Run / Recommended Mentor, Requests / Allocations, Availability / Booking, Sessions, Session Feedback, Mentor Review, Notification Outbox and Audit Log.
AlumniConnect Product Requirements Document AC-PRD-001 (13 pages)	FR-05 to FR-10 for skills, profiles, discovery, recommendation score dimensions and explanations; FR-11 / FR-12 request and allocation; FR-13 / FR-14 booking; FR-16 to FR-20 sessions, feedback, reviews and outbox; FR-22 analytics; FR-23 Redis.
AlumniConnect Business Rules Catalogue (31 pages)	Business rules, SQL constraints, triggers, procedures, transactions, indexes, review validation, history retention, security and test expectations.
AlumniConnect Workflow and State Diagrams (25 pages)	Request, allocation, booking and session status lifecycles; transaction boundaries; booking checks; feedback / review distinction; notification outbox.
AlumniConnect Use Case Model (20 pages)	Student, mentor and administrator interactions; discovery, requests, scheduling, session outcomes, reviews and administrative reports.

Supplied source	Relevant content used in this catalogue
Alumni Mentor Allocation and Student Mentorship Management System - earlier DBMS Mini Project (12-table schema)	Legacy table names and attributes; match_score and allocation_reason; example DBMS questions; data volumes; status and cardinality baseline.
DBMS Unit 2 lecture slides - Additional Basic Operations / Advanced SQL	SELECT, joins, aggregates, nested / correlated queries, views, triggers, functions and procedures.
DBMS UNIT 2(5).pdf - Triggers and Procedures lecture material	Trigger purpose and BEFORE/AFTER INSERT/UPDATE/DELETE syntax, FOR EACH ROW, NEW/OLD values, SIGNAL SQLSTATE; stored procedure syntax and IN/OUT/INOUT parameter conventions.
DBMS_U1 course material / course outline	Unit 1 relational foundations; Unit 2 advanced SQL; Unit 3 normalization and query optimization; Unit 4 transactions, concurrency, Redis and Neo4j.
DBMS Unit 4 lecture material - Transactions / ACID / Concurrency	ACID boundaries, row-level locking, isolation, lost updates, deadlocks and strict/two-phase locking concepts used in the transaction guidance.
AlumniConnect Transaction and Consistency Design (32 pages)	ACID boundaries; one transaction for business state plus outbox; parent-row locking for capacity / overlap serialization; booking conflict and rollback tests; cross-database consistency and idempotent worker behaviour.
AlumniConnect Neo4j Graph Design (32 pages)	Graph projection as a specialised recommendation data model; graph is not a source of truth; candidate discovery and evidence returned to SQL / application; MySQL final eligibility checks.
AlumniConnect Redis Design Specification (24 pages)	Redis cache structures and cache-aside behavior; recommendation and profile cache invalidation after MySQL commits; Redis is never booking authority; TTL and retry behavior.

Source page pointers (for quick review)

- Blueprint: project vision / roles pp. 1-3; database responsibilities and entity catalogue approx. pp. 7-13; Neo4j recommendation model pp. 14-19; workflows pp. 20-27; DBMS showcase and query optimisation pp. 28-34.
- Application Architecture: module catalogue p. 6; workflows and relational domains pp. 7-8; Redis / Neo4j model and recommendation dimensions pp. 9-10; outbox and cross-database consistency p. 11; DBMS unit mapping p. 16.
- Conceptual ER Diagram: scope and notation p. 2; identity and skills diagrams pp. 3-4; entity catalogue pp. 9-11; relationships pp. 12-14.
- PRD: requirements FR-05 to FR-10 pp. 5; FR-11 to FR-18 pp. 5-7; notification/outbox/admin analytics/Redis requirements p. 7; non-functional traceability and cross-database consistency p. 10.
- DBMS Unit 2(5).pdf: trigger concept and syntax around PDF pp. 213-217; validation / OLD-NEW examples around pp. 220-223; stored procedure syntax and parameter modes around pp. 242-245.
- DBMS Unit 4 transaction material: ACID section around PDF p. 26; locking and concurrency examples in the transaction/concurrency chapters.
- Transaction and Consistency Design: ACID and invariants pp. 5-6; locking / isolation / concurrency and transaction patterns in the middle sections; document traceability p. 29.
- Neo4j Graph Design: recommendation scope and MySQL boundary p. 4; graph entity/relationship schema and candidate evidence in the graph model; source and design summary p. 30.
- Redis Design Specification: cache invalidation and recommendation-cache rules across the design sections; source traceability in Appendix C / p. 24.

Appendix B. SQL execution checklist

Use this checklist when installing the catalogue in the agreed MySQL development schema. Complete each step against the team's final DDL; the code examples are implementation templates, not a substitute for schema review.

- Confirm MySQL version (8.x) and that all referenced tables and columns exist in the final schema.
- Confirm that all tables use InnoDB and foreign keys are enabled.
- Review existing indexes before creating the recommended ones; duplicate indexes waste space and slow writes.
- Run views, functions, procedures and triggers in a development schema first.
- Use DELIMITER $$ when creating stored routines in MySQL Workbench / CLI, then restore DELIMITER ;.
- Test procedures with both success and forced-failure cases; inspect all tables after ROLLBACK.
- Test two separate sessions for slot booking and mentor-capacity races.
- Use application prepared statements for all user input; SQL user variables in this PDF are demonstration placeholders.
- Keep Redis and Neo4j as projections/caches. A query may use them to discover or accelerate, but MySQL validates every business action.
- Record actual EXPLAIN / EXPLAIN ANALYZE output and benchmark results; do not claim a performance improvement without measurements.

Evidence to retain for the DBMS demonstration

- A schema snapshot showing table definitions, primary keys, foreign keys, CHECK constraints and indexes.
- Successful and rejected procedure-call examples, including the state of affected rows after a forced rollback.
- A two-connection booking race showing that only one transaction can confirm the same slot.
- Representative EXPLAIN / EXPLAIN ANALYZE plans before and after indexing, with the dataset size and test environment recorded.
- A short outbox-worker retry demonstration proving that Redis / Neo4j or notification-delivery failures do not undo a committed MySQL business change.

Final implementation reminder Before using this catalogue as executable production SQL, reconcile the Section 3 contract with the approved physical schema. The most important invariants to preserve are request-to-allocation uniqueness, slot exclusivity, no overlapping confirmed bookings, completed-session review eligibility and atomic outbox writes.